

Grile de Machine Learning pentru examen

teorie esentiala + intrebari tip grila explicate

Document de recapitulare pentru partea de Machine Learning din Inteligenta Artificiala.

Include formule, capcane frecvente si grile similare ca stil cu intrebarile de examen: cate o idee testata, 4 variante, raspuns si explicatie.

Cuprins

1 Cum se foloseste documentul	2
2 Teoria esentiala pe capitole	2
2.1 Invatare automata: ideea principala	2
2.2 Tipuri de invatare	2
2.3 Clasificare vs regresie	2
2.4 Antrenare, validare, testare	3
2.5 Eroare, generalizare, underfitting, overfitting	3
2.6 Matricea de confuzie si metrici	3
2.7 Bayes Naiv	3
2.8 k-NN	4
2.9 Preprocesare: standardizare, normalizare, BoW	4
2.10Clasificator liniar, SVM multiclass, Softmax	5
2.11Regularizare	5
2.12Gradient descent	5
2.13Regresie liniara si Ridge	5
2.14Metode kernel si matricea Gram	6
2.15Rețele neuronale si backpropagation	6
3 Grile explicate	6
4 Mini-exercitii de calcul, stil examen	16
5 Capcane frecvente in grile	16
6 Checklist final de invatat	17
7 Surse folosite	17

1 Cum se foloseste documentul

Pentru examen, nu invata formulele izolat. La fiecare formula trebuie sa stii: **ce reprezinta fiecare termen, cand se aplica, ce se intampla in cazuri limita si ce capcana poate aparea in grile.**

Ordinea recomandata:

1. Citeste teoria scurta de la fiecare capitol.
2. Rezolva grilele fara sa te uiti la raspuns.
3. Verifica explicatia, nu doar litera raspunsului.
4. Marcheaza grilele unde ai gresit: acelea arata exact teoria care trebuie repetata.

2 Teoria esentiala pe capitole

2.1 Invatare automata: ideea principala

Machine Learning inseamna sa invatam o functie care mapeaza un input X la un output Y pe baza datelor.

$$f : X \rightarrow Y$$

Exemple:

- input: imagine, output: clasa imaginii;
- input: text, output: spam / non-spam;
- input: caracteristici ale unei locuinte, output: pret.

Orice algoritm ML are trei componente mari: **reprezentare/model, functie de evaluare/pierdere, optimizare**. In grile, multe intrebari cer sa identifici care componenta este modificata.

2.2 Tipuri de invatare

Tip	Ce inseamna
Supervizata	Avem exemple etichetate (x_i, y_i) . Include clasificare si regresie.
Nesupervizata	Avem doar exemple x_i , fara etichete. Exemple: clustering, reducerea dimensionalitatii.
Semi-supervizata	Avem cateva exemple etichetate si multe neetichetate.
Ranforsata	Agentul invata din recompense primite dupa actiuni. Datele sunt secventiale, nu i.i.d.
Activa	Modelul alege ce exemple sa fie etichetate.
Transfer	Folosim un model invatat pe o problema/domeniu pentru alta problema.

2.3 Clasificare vs regresie

- **Clasificare**: output discret, de exemplu clasa $\{0, 1, 2\}$.
- **Regresie**: output numeric continuu, de exemplu pret, temperatura, scor real.

2.4 Antrenare, validare, testare

- **Train:** folosit pentru invatarea parametrilor.
- **Validation:** folosit pentru alegerea hiperparametrilor: k , C , λ , learning rate etc.
- **Test:** folosit doar pentru evaluarea finala.

Capcana: daca folosesti datele de test ca sa alegi modelul, faci **data leakage**. Scorul final nu mai estimeaza corect generalizarea.

2.5 Eroare, generalizare, underfitting, overfitting

- **Eroarea empirica:** eroarea calculata pe datele disponibile.
- **Eroarea de generalizare:** eroarea asteptata pe date nevazute.
- **Underfitting:** model prea simplu, eroare mare pe train si test, bias mare.
- **Overfitting:** model prea complex, eroare mica pe train si mare pe test, variance mare.

Concept	Simptom	Solutie tipica
Bias mare	modelul nu invata nici train	model mai complex, feature-uri mai bune
Variance mare	train bun, test prost	regularizare, mai multe date, augmentare, model mai simplu

2.6 Matricea de confuzie si metrici

Pentru clasificare binara:

	Prezis pozitiv	Prezis negativ
Real pozitiv	TP	FN
Real negativ	FP	TN

Formule:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Precision = \frac{TP}{TP + FP}, \quad Recall = \frac{TP}{TP + FN}$$

$$F_1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$$

$$F_\beta = (1 + \beta^2) \cdot \frac{Precision \cdot Recall}{\beta^2 \cdot Precision + Recall}$$

$\beta > 1$ favorizeaza **recall**; $\beta < 1$ favorizeaza **precision**; $\beta = 1$ le trateaza echilibrat.

2.7 Bayes Naiv

Regula Bayes:

$$P(c | X) = \frac{P(X | c)P(c)}{P(X)}$$

La clasificare, $P(X)$ este acelasi pentru toate clasele, deci alegem:

$$\hat{c} = \arg \max_c P(c) \prod_{i=1}^n P(x_i | c)$$

In practica se folosesc logaritmi:

$$\hat{c} = \arg \max_c \left[\log P(c) + \sum_{i=1}^n \log P(x_i | c) \right]$$

Presupunerea “naiva”: attributele x_i sunt independente conditionat de clasa c . Nu in-seamna ca sunt independente in realitate, ci ca modelul le trateaza asa pentru calcul.

Tipuri uzuale:

- **MultinomialNB**: frecvente/count-uri, BoW, pixeli discretizati.
- **BernoulliNB**: attribute binare, prezenta/absenta.
- **GaussianNB**: attribute continue modelate cu distributii Gaussiene.

Smoothing Laplace: daca apare o valoare necunoscuta in test, fara smoothing putem avea probabilitate zero. Cu smoothing:

$$P(x = v | c) = \frac{N_{c,v} + \alpha}{N_c + \alpha K}$$

unde K este numarul de valori posibile pentru atribut.

2.8 k-NN

Algoritm:

1. Pentru exemplul de test x , calculam distantele pana la exemplele de antrenare.
2. Alegem cei mai apropiati k vecini.
3. Pentru clasificare: vot majoritar. Pentru regresie: media etichetelor.

Distantelor importante:

$$L_1(x, y) = \sum_i |x_i - y_i|, \quad L_2(x, y) = \sqrt{\sum_i (x_i - y_i)^2}$$

$$L_p(x, y) = \left(\sum_i |x_i - y_i|^p \right)^{1/p}$$

k mic inseamna model mai sensibil la zgomot, deci variance mai mare. k mare netezeste frontiera de decizie, dar poate produce bias mai mare.

Normalizarea conteaza: daca o componenta are valori mult mai mari decat alta, domina distanta.

2.9 Preprocesare: standardizare, normalizare, BoW

Standardizare:

$$x_{scaled} = \frac{x - \mu}{\sigma}$$

unde μ si σ se calculeaza pe **train**, apoi se aplica pe validation/test.

Normalizare L_1 si L_2 :

$$x_{L1} = \frac{x}{\|x\|_1}, \quad x_{L2} = \frac{x}{\|x\|_2}$$

Bag-of-Words:

- construim vocabularul din train;
- fiecare document devine vector de frecvente;
- ordinea cuvintelor se pierde.

2.10 Clasificator liniar, SVM multiclass, Softmax

Un clasificator liniar produce scoruri:

$$s = Wx + b$$

unde fiecare componenta s_j este scorul clasei j .

SVM multiclass / hinge loss: pentru exemplul i cu eticheta corecta y_i :

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + \Delta)$$

De obicei $\Delta = 1$.

Softmax transforma scorurile in probabilitati:

$$P(y = j | x) = \frac{e^{s_j}}{\sum_k e^{s_k}}$$

Loss-ul pentru clasa corecta y :

$$L_i = -\log P(y_i | x_i)$$

SVM cere ca scorul clasei corecte sa fie cu o marja peste celelalte. Softmax incurajeaza probabilitatea clasei corecte sa fie cat mai aproape de 1.

2.11 Regularizare

Loss total:

$$L = L_{data} + \lambda R(W)$$

Pentru L_2 :

$$R(W) = \sum_j W_j^2$$

- λ mare: regularizare puternica, model mai simplu, risc de underfitting.
- λ mic: model mai flexibil, risc de overfitting.

2.12 Gradient descent

Actualizarea parametrilor:

$$W \leftarrow W - \eta \nabla_W L$$

unde η este learning rate.

- learning rate prea mare: loss-ul poate oscila/diverge;
- learning rate prea mic: invatarea este foarte lenta;
- gradientul indica directia de crestere maxima a functiei; de aceea scadem gradientul pentru minimizare.

2.13 Regresie liniara si Ridge

Regresia liniara cauta:

$$g(x) = w^T x$$

care minimizeaza eroarea patratica. In forma matriceala:

$$L(w) = \|Xw - y\|^2$$

Solutia normala, cand inversa exista:

$$w = (X^T X)^{-1} X^T y$$

Ridge adauga regularizare:

$$w = (X^T X + \lambda I)^{-1} X^T y$$

2.14 Metode kernel si matricea Gram

Metodele kernel permit separari neliniare prin mapare implicita intr-un spatiu mai mare.

$$k(x, z) = \langle \phi(x), \phi(z) \rangle$$

Matricea Gram pentru train are forma:

$$K_{ij} = k(x_i, x_j)$$

Daca sunt n exemple de antrenare, atunci $K \in \mathbb{R}^{n \times n}$.

Kerneluri frecvente:

- liniar: $k(x, z) = x^T z$;
- polinomial: $k(x, z) = (x^T z + c)^d$;
- RBF/Gaussian: $k(x, z) = \exp(-\gamma \|x - z\|^2)$.

2.15 Retele neuronale si backpropagation

O retea neuronală combina straturi liniare cu functii de activare:

$$h = \sigma(W_1 x + b_1), \quad s = W_2 h + b_2$$

Backpropagation aplica regula lantului pentru a calcula gradientii eficient.

Functii de activare:

- sigmoid: valori in $(0, 1)$, dar poate satura si omoara gradientii;
- tanh: valori in $(-1, 1)$, centrat in zero;
- ReLU: $\max(0, x)$, simpla si folosita des;
- Leaky ReLU: lasa un gradient mic si pentru valori negative.

3 Grile explicate

Grila 1. Care afirmatie descrie cel mai bine invatarea supervizata?

- A. Avem doar date fara etichete si vrem sa gasim grupuri.
- B. Avem perechi (x, y) si vrem sa invatam o functie care prezice y din x .
- C. Agentul invata exclusiv din recompense.
- D. Modelul alege singur ce date sa eticheteze.

Raspuns: B. Invatarea supervizata foloseste exemple etichetate. A descrie nesupervizata, C descrie ranforsata, D descrie invatarea activa.

Grila 2. Care este diferenta corecta intre clasificare si regresie?

- A. Clasificarea are output continuu, regresia are output discret.
- B. Clasificarea are output discret, regresia are output numeric continuu.
- C. Ambele au mereu output binar.
- D. Regresia nu foloseste date de antrenare.

Raspuns: B. Clasele sunt valori discrete, de exemplu 0/1/2. Regresia prezice valori reale, de exemplu pretul unei case.

Grila 3. Un model are eroare mare pe train si eroare mare pe test. Ce se intampla cel mai probabil?

- A. Overfitting.
- B. Underfitting.
- C. Variance mare si bias mic.
- D. Data leakage.

Raspuns: B. Daca modelul nu invata bine nici datele de antrenare, este prea simplu sau are feature-uri slabe: bias mare, underfitting.

Grila 4. Un model are acuratete 99% pe train si 65% pe test. Interpretarea cea mai probabila este:

- A. Underfitting.
- B. Overfitting.
- C. Bias mare.
- D. Modelul este sigur optim.

Raspuns: B. Train foarte bun si test mult mai slab indica memorare/zgomot invatat: variance mare, overfitting.

Grila 5. Pentru clasificare binara avem $TP = 40$, $TN = 50$, $FP = 10$, $FN = 20$. Care este acuratetea?

- A. $40/(40 + 10)$
- B. $40/(40 + 20)$
- C. $(40 + 50)/(40 + 50 + 10 + 20)$
- D. $(10 + 20)/(40 + 50 + 10 + 20)$

Raspuns: C. Acuratetea este proportia predictiilor corecte: $(TP + TN)/(TP + TN + FP + FN) = 90/120 = 0.75$.

Grila 6. Cu aceleasi valori $TP = 40$, $FP = 10$, $FN = 20$, precision este:

- A. $40/(40 + 10)$
- B. $40/(40 + 20)$
- C. $50/(50 + 10)$
- D. $40/(40 + 50)$

Raspuns: A. Precision raspunde la intrebarea: dintre cele prezise pozitive, cate sunt corecte? Formula este $TP/(TP + FP)$.

Grila 7. Pentru F_β , ce se favorizeaza cand $\beta > 1$?

- A. Precision.
- B. Recall.

- C. Acuratetea.
- D. Specificitatea.

Raspuns: B. $\beta > 1$ pune greutate mai mare pe recall. $\beta < 1$ pune greutate mai mare pe precision.

Grila 8. In Bayes Naiv, formula folosita pentru decizie este de obicei:

- A. $\arg \max_c P(c) \prod_i P(x_i|c)$
- B. $\arg \min_c P(c) \prod_i P(x_i|c)$
- C. $\arg \max_c P(x_i)$, fara clase
- D. $\arg \max_c \sum_i x_i$, fara probabilitati

Raspuns: A. Alegem clasa cu probabilitatea posteroara maxima. Factorul $P(X)$ se ignora deoarece este acelasi pentru toate clasele.

Grila 9. De ce se folosesc logaritmi in Bayes Naiv?

- A. Ca sa transformam produsul de probabilitati in suma si sa evitam underflow numeric.
- B. Ca sa schimbam clasificarea in regresie.
- C. Ca sa eliminam priorul $P(c)$.
- D. Ca sa facem attributele dependente.

Raspuns: A. Produsul multor probabilitati mici poate deveni numeric aproape zero. Logaritmul transforma produsul in suma.

Grila 10. Ce problema rezolva smoothing-ul Laplace in Naive Bayes?

- A. Valori de probabilitate zero pentru attribute neobservate.
- B. Overfitting in retele neuronale.
- C. Alegerea lui k in k-NN.
- D. Dimensiunea matricei Gram.

Raspuns: A. Fara smoothing, daca o valoare nu a aparut in train pentru o clasa, $P(x|c) = 0$ si anuleaza tot produsul.

Grila 11. Pentru date continue, Bayes Naiv poate fi aplicat pe pixeli MNIST dupa:

- A. discretizare in intervale/bin-uri;
- B. stergerea tuturor pixelilor;
- C. inlocuirea etichetelor cu distante;
- D. calculul matricei Gram.

Raspuns: A. In laborator, valorile continue ale pixelilor sunt transformate in valori discrete prin histograma/bin-uri.

Grila 12. In k-NN, ce inseamna $k = 1$?

- A. Alegem eticheta celui mai apropiat exemplu de train.
- B. Facem media tuturor exemplelor.
- C. Folosim un singur atribut.
- D. Folosim un singur epoch.

Raspuns: A. 1-NN clasifica dupa cel mai apropiat vecin. Este foarte sensibil la zgomot.

Grila 13. Pentru $x = (1, 2)$ si $y = (4, 6)$, distanta L_1 este:

- A. $\sqrt{3^2 + 4^2} = 5$
- B. $|1 - 4| + |2 - 6| = 7$
- C. $3 \cdot 4 = 12$
- D. $1 + 2 + 4 + 6 = 13$

Raspuns: B. L_1 inseamna suma diferentelor absolute: $3 + 4 = 7$.

Grila 14. Pentru $x = (1, 2)$ si $y = (4, 6)$, distanta L_2 este:

- A. 7
- B. 5
- C. 25
- D. 3

Raspuns: B. $L_2 = \sqrt{(1 - 4)^2 + (2 - 6)^2} = \sqrt{9 + 16} = 5$.

Grila 15. Ce efect are cresterea lui k in k-NN?

- A. Frontiera devine de obicei mai neteda, variance scade, bias poate creste.
- B. Modelul devine mereu perfect.
- C. Nu se schimba nimic.
- D. Nu mai este nevoie de distanta.

Raspuns: A. Un k mai mare regularizeaza votul, dar poate netezi prea mult si poate rata structuri locale.

Grila 16. De ce normalizarea este importanta pentru k-NN?

- A. Pentru ca distantele sunt afectate de scara atributelor.
- B. Pentru ca transforma k-NN in SVM.
- C. Pentru ca elimina necesitatea train-ului.
- D. Pentru ca face toate etichetele egale.

Raspuns: A. Daca o trasatura are valori 0-10000 si alta 0-1, prima domina distanta.

Grila 17. Ce este “blestemul dimensionalitatii” pentru k-NN?

- A. In dimensiuni mari, distantele devin mai putin informative si sunt necesare mult mai multe date.
- B. k-NN nu poate folosi mai mult de doua attribute.
- C. L1 nu se mai poate calcula.
- D. Setul de train dispare.

Raspuns: A. In spatii cu multe dimensiuni, exemplele devin rare, iar vecinatatea locala este mai greu de definit.

Grila 18. In bag-of-words, un document este reprezentat prin:

- A. frecvente ale cuvintelor din vocabular;
- B. ordinea exacta a pixelilor;
- C. gradientii retelei;
- D. matricea de confuzie.

Raspuns: A. BoW foloseste un vocabular si numara aparitiile cuvintelor. Ordinea cuvintelor este de regula pierduta.

Grila 19. Pentru standardizare, media si deviatia standard trebuie calculate pe:

- A. train, apoi aplicate pe validation/test;
- B. test, apoi aplicate pe train;
- C. train+test combinat, ca sa fie mai precis;
- D. fiecare exemplu separat, mereu.

Raspuns: A. Statistica se invata doar din train. Folosirea testului in preprocesare produce data leakage.

Grila 20. Pentru un clasificator liniar multiclass, scorurile sunt $s = [3.2, 5.1, -1.7]$ si clasa corecta este prima. Hinge loss cu $\Delta = 1$ este:

- A. 0
- B. 2.9
- C. 5.1
- D. 3.2

Raspuns: B. Se calculeaza $\max(0, 5.1 - 3.2 + 1) + \max(0, -1.7 - 3.2 + 1) = 2.9 + 0 = 2.9$.

Grila 21. Cand SVM multiclass are loss zero pentru un exemplu?

- A. Cand scorul clasei corecte este mai mare decat fiecare scor gresit cu cel putin marja Δ .
- B. Cand toate scorurile sunt egale.

- C. Cand scorul clasei corecte este cel mai mic.
- D. Cand $k = 1$.

Raspuns: A. Hinge loss penalizeaza scorurile gresite care sunt prea aproape sau peste scorul corect.

Grila 22. Softmax transforma scorurile in:

- A. probabilitati care insumeaza 1;
- B. distante Manhattan;
- C. valori binare doar 0 sau 1;
- D. indici de bin-uri.

Raspuns: A. Softmax aplica exponentiala si normalizeaza prin suma exponentialiei scorurilor.

Grila 23. Pentru Softmax, loss-ul clasei corecte este $-\log(p_{corect})$. Daca $p_{corect} = 1$, loss-ul este:

- A. 0
- B. 1
- C. $+\infty$
- D. -1

Raspuns: A. $-\log(1) = 0$. Loss-ul minim este 0. Daca probabilitatea clasei corecte tinde la 0, loss-ul tinde la infinit.

Grila 24. Care este diferenta intuitiva dintre SVM loss si Softmax loss?

- A. SVM verifica marje, Softmax lucreaza cu probabilitati normalizate.
- B. SVM este doar pentru regresie, Softmax doar pentru clustering.
- C. SVM nu foloseste scoruri.
- D. Softmax nu are functie de pierdere.

Raspuns: A. SVM vrea scor corect suficient de mare fata de celelalte. Softmax vrea probabilitate mare pentru clasa corecta.

Grila 25. Ce face regularizarea L_2 ?

- A. Penalizeaza ponderile mari si poate reduce overfitting-ul.
- B. Mareste mereu toate ponderile.
- C. Inlocuieste complet loss-ul de date.
- D. Schimba clasificarea in regresie.

Raspuns: A. Termenul $\lambda \sum W_j^2$ descurajeaza ponderile foarte mari si reduce complexitatea efectiva.

Grila 26. Daca λ este foarte mare in regularizare, ce risc apare?

- A. Underfitting.
- B. Overfitting garantat.
- C. Distantele devin imposibil de calculat.
- D. Nu mai exista clase.

Raspuns: A. Regularizarea prea puternica forteaza modelul sa fie prea simplu.

Grila 27. In gradient descent, actualizarea corecta pentru minimizare este:

- A. $W \leftarrow W - \eta \nabla_W L$
- B. $W \leftarrow W + \eta \nabla_W L$
- C. $W \leftarrow W / \eta$
- D. $W \leftarrow \nabla_W L$

Raspuns: A. Gradientul indica directia de crestere maxima a loss-ului. Pentru minimizare mergem in sens opus.

Grila 28. Daca learning rate este prea mare, ce se poate intampla?

- A. Loss-ul poate oscila sau diverge.
- B. Modelul devine automat mai precis.
- C. Gradientul devine mereu zero.
- D. Softmax se transforma in SVM.

Raspuns: A. Pasi prea mari pot sari peste minim si pot destabiliza optimizarea.

Grila 29. Numeric, derivata aproximativa pe o dimensiune poate fi calculata ca:

- A. $\frac{L(W+h)-L(W)}{h}$
- B. $L(W+h) + L(W)$
- C. $h/L(W)$
- D. $L(W)^2$

Raspuns: A. Aceasta este diferenta finita inainte, folosita pentru verificarea gradientului.

Grila 30. In regresie liniara, solutia normala este:

- A. $w = (X^T X)^{-1} X^T y$
- B. $w = X + y$
- C. $w = (X X^T)^{-1}$ mereu
- D. $w = \arg \max P(c|X)$

Raspuns: A. Aceasta vine din minimizarea erorii patratice si anulara gradientului.

Grila 31. De ce folosim regresia Ridge?

- A. Pentru regularizare si stabilitate cand $X^T X$ este prost conditionata sau neinversabila.
- B. Pentru a elimina toate feature-urile.
- C. Pentru a obtine distante L1.
- D. Pentru a transforma datele in etichete discrete.

Raspuns: A. Ridge adauga λI , ceea ce stabilizeaza inversarea si penalizeaza ponderile mari.

Grila 32. Matricea Gram pentru n exemple de antrenare are dimensiunea:

- A. $n \times n$
- B. $d \times d$ mereu
- C. $1 \times n$
- D. $n \times 1$

Raspuns: A. $K_{ij} = k(x_i, x_j)$ contine similaritatea fiecarei perechi de exemple de train.

Grila 33. Ce spune "kernel trick"?

- A. Putem inlocui produsul scalar in spatiul transformat cu o functie kernel, fara sa calculam explicit transformarea $\phi(x)$.
- B. Putem ignora complet datele de train.
- C. Putem folosi doar clasificatori liniari in spatiul original.
- D. Putem calcula acuratetea fara predictii.

Raspuns: A. Kernelul calculeaza implicit produsul scalar intr-un spatiu de dimensiune posibil mare.

Grila 34. Kernelul RBF are forma generala:

- A. $\exp(-\gamma \|x - z\|^2)$
- B. $x + z$
- C. $\sum_i |x_i - z_i|$
- D. $\log P(c)$

Raspuns: A. RBF masoara similaritatea printr-o exponentiala a distantei patratice.

Grila 35. De ce metodele kernel ajuta la probleme neliniare?

- A. Pot corespunde unei mapari intr-un spatiu unde separarea devine liniara.
- B. Sterg clasele gresite.
- C. Inlocuiesc train-ul cu test-ul.

D. Folosesc numai $k = 1$.

Raspuns: A. Relatii neliniare in spatiul initial pot deveni liniare dupa o scufundare in spatiu de dimensiune mai mare.

Grila 36. Functia XOR nu poate fi invatata de un clasificator liniar simplu deoarece:

- A. clasele nu sunt separabile liniar in spatiul original;
- B. are prea putine exemple;
- C. are doar doua clase;
- D. foloseste valori binare.

Raspuns: A. XOR este exemplul clasic unde o dreapta nu poate separa corect clasele. Retele neuronale sau kerneluri pot rezolva problema.

Grila 37. Backpropagation reprezinta:

- A. aplicarea regulii lantului pentru calculul gradientilor in retea;
- B. vot majoritar intre vecini;
- C. discretizarea pixelilor;
- D. calculul matricei de confuzie.

Raspuns: A. Backprop calculeaza eficient gradientii parametrilor folosind graf computational si regula de inlantuire.

Grila 38. Care este o problema a functiei sigmoid?

- A. Se poate satura si gradientii devin foarte mici.
- B. Nu produce niciodata valori intre 0 si 1.
- C. Nu este neliniara.
- D. Este identica cu ReLU.

Raspuns: A. Pentru valori foarte mari sau foarte mici, derivata sigmoidei este aproape zero, deci invatarea devine lenta.

Grila 39. ReLU este definita ca:

- A. $\max(0, x)$
- B. $1/(1 + e^{-x})$
- C. $e^x / \sum e^x$
- D. $|x - y|$

Raspuns: A. ReLU pastreaza valorile pozitive si pune la zero valorile negative.

Grila 40. Ce inseamna antrenare end-to-end in deep learning?

- A. Modelul invata direct de la input brut la output, inclusiv reprezentarile intermediare.
- B. Feature-urile sunt obligatoriu scrise manual.
- C. Nu exista functie de pierdere.
- D. Datele de test sunt folosite la antrenare.

Raspuns: A. In loc sa extragem manual feature-uri, retea invata reprezentarile utile impreuna cu clasificatorul final.

Grila 41. Pentru C clase, strategia SVM one-vs-one antreneaza:

- A. $C(C - 1)/2$ clasificatori binari.
- B. C clasificatori mereu.
- C. $2C$ clasificatori.
- D. un singur clasificator doar daca $C > 2$.

Raspuns: A. Se antreneaza cate un clasificator pentru fiecare pereche de clase.

Grila 42. In SVM, un C foarte mare inseamna de obicei:

- A. penalizare mai mare pentru erorile de clasificare, risc mai mare de overfitting;
- B. regularizare mai puternica spre ponderi zero;
- C. model complet nesupervizat;
- D. eliminarea vectorilor suport.

Raspuns: A. C mare forteaza modelul sa clasifice corect train-ul, chiar cu margine mai ingusta.

Grila 43. In k-NN pentru regresie, predictia este de obicei:

- A. media valorilor target ale celor k vecini;
- B. clasa majoritara;
- C. log-probabilitatea clasei;
- D. gradientul loss-ului.

Raspuns: A. Pentru regresie, output-ul este numeric, deci se foloseste media sau media ponderata a vecinilor.

Grila 44. Ce inseamna un epoch in antrenarea unei retele?

- A. O parcurgere completa a setului de antrenare.
- B. O singura componenta a vectorului.
- C. O clasa din dataset.
- D. Un hiperparametru specific k-NN.

Raspuns: A. Un epoch inseamna ca modelul a vazut toate exemplele de train o data.

Grila 45. Care este rolul setului de validare?

- A. Alegerea hiperparametrilor si compararea modelelor inainte de testul final.
- B. Antrenarea directa a tuturor parametrilor.
- C. Raportarea singurului scor final fara risc de bias.
- D. Discretizarea automata a pixelilor.

Raspuns: A. Validation ajuta la selectia modelului. Testul trebuie pastrat pentru evaluarea finala.

4 Mini-exercitii de calcul, stil examen

Exercitiul 1: Naive Bayes cu bin-uri

Avem train: $(160, F), (165, F), (155, F), (172, F), (175, B), (180, B), (177, B), (190, B)$. Bin-uri: 150-160, 161-170, 171-180, 181-190. Pentru o persoana de 178 cm, determinati clasa cu Bayes Naiv fara smoothing.

Rezolvare scurta.

- Priori: $P(F) = 4/8 = 1/2$, $P(B) = 4/8 = 1/2$.
- 178 este in bin-ul 171-180.
- Pentru F: in bin apar valorile 172 dintre cele 4 fete, deci $P(bin|F) = 1/4$.
- Pentru B: in bin apar 175, 180, 177 dintre cei 4 baieti, deci $P(bin|B) = 3/4$.
- Scor F: $1/2 \cdot 1/4 = 1/8$. Scor B: $1/2 \cdot 3/4 = 3/8$.

Predictie: B.

Exercitiul 2: 3-NN cu L1

Avem trei vecini cei mai apropiati cu etichetele $[1, 1, 2]$. Ce prezice modelul?

Rezolvare: clasa 1 are doua voturi, clasa 2 are un vot. Predictie: 1.

Exercitiul 3: Hinge loss

Scoruri $s = [2, 4, 1]$, clasa corecta este prima, $\Delta = 1$.

$$L = \max(0, 4 - 2 + 1) + \max(0, 1 - 2 + 1) = 3 + 0 = 3.$$

Exercitiul 4: Dimensiunea matricei kernel

Daca X_{train} are 100 exemple si 784 feature-uri, matricea Gram de train are dimensiunea 100×100 , nu 784×784 .

5 Capcane frecvente in grile

1. **Accuracy vs precision/recall:** accuracy nu spune tot cand clasele sunt dezechilibrate.
2. **Bayes Naiv:** $P(X)$ nu conteaza la compararea claselor, dar priorul $P(c)$ conteaza.
3. **Smoothing:** daca apare probabilitate zero, clasa poate fi anulata artificial.
4. **k-NN:** nu exista antrenare parametrica propriu-zisa; costul mare apare la testare.
5. **Standardizare:** fit pe train, transform pe validation/test.

6. **SVM vs Softmax**: SVM este pe marja; Softmax este pe probabilitate.
7. **Gradient**: pentru minimizare scazi gradientul, nu il aduni.
8. **Kernel**: matricea Gram este intre exemple, nu intre feature-uri.
9. **Ridge**: adauga λI , nu scade λI .
10. **Retele**: backprop nu este un model separat, ci metoda de calcul a gradientilor.

6 Checklist final de invatat

- paradigme ML;
- train/validation/test;
- generalizare;
- bias/variance;
- underfitting/overfitting;
- matrice confuzie;
- accuracy, precision, recall, F1, F_β ;
- Bayes Naiv si log-probabilitati;
- smoothing Laplace;
- discretizare in bin-uri;
- k-NN clasificare si regresie;
- distante L1, L2, Hamming;
- efectul lui k ;
- normalizare L1/L2 si standardizare;
- BoW;
- scoruri liniare $Wx + b$;
- SVM hinge loss;
- Softmax + cross entropy;
- regularizare L_2 ;
- gradient descent;
- regresie liniara;
- Ridge;
- kernel trick;
- matrice Gram;
- RBF/polinomial/liniar;
- perceptron si XOR;
- retele neuronale;
- activari sigmoid/tanh/ReLU;
- backpropagation;
- epoch, batch, learning rate.

7 Surse folosite

Documentul a fost construit ca set original de recapitulare si grile, pe baza temelor din materialele de curs/laborator incarcate: concepte generale ML, Naive Bayes, k-NN, SVM, kernel/Ridge, gradient descent si retele neuronale. Site-ul indicat a fost folosit ca reper de stil pentru grile, dar intrebarile de aici sunt generate nou si explicate.